

Thoughts on RNG - Using Bash Scripts

Frank Jung

01 Feb 2026

Contents

Testing Numeric Sequences	2
Example 5.1 in Gibbons and Chakraborti (2003), p.98.	2
Example in Bartels (1982)	3
Random Number Generators that can be used in Bash scripts	5
bash RANDOM variable	5
awk rand()	7
urandom device	8
Some final thoughts	10



One of the first exercises given to me as a mathematics student was to write a random number generator (RNG). This turned out to be not so easy. Test sequences cycled quickly or were too predictable or were not evenly distributed. Typically when we talk of RNG's we are describing *pseudorandom* number generators. Nowadays, we have a many programs that will generate *pseudorandom* numbers.

Where are random numbers used? As a developer they were rarely required. Recently, however we've seen them appear in more and more places - it seems they are everywhere!

In DevOps, I've used RNG's for creating message payloads of arbitrary size, and for file or directory names. These values are often created using scripts written in bash. This first article will explore three simple RNG's that can be run from bash. It is not an exhaustive list, as there are others such as jot that is also easy to use from bash. However, the three described here are likely to already be installed on your Linux box.

The three RNG's being evaluated here are:

- Bash `RANDOM` variable
- Awk `rand()` function
- `/dev/urandom` device

Words of caution:

None of these tools are suitable for generating passwords or for cryptography.

Source and test data for this article can be found [here](#).

In future articles we will take a closer look at random values used in testing code and how they can be used for model simulation in statistics.

Testing Numeric Sequences

This discussion will not test the programs for randomness. Instead, we are going to evaluate a short list of 1000 values generated by each RNG. To ease comparison the values will be scaled to the range $[0, 1)$. They are rounded to two decimal places and the list of values is formatted as `0.nn`.

There are many tests that can be applied to sequences to check for randomness. Here we are only looking at one: the Bartels Rank test. Its limitations and those of other tests are described [here](#). I've chosen this test as it is relatively easy to understand and interpret. Rather than comparing the magnitude of each observation with its preceding sample, Bartels Rank test, ranks all the samples from the smallest to the largest. The rank is the corresponding sequential number in the list of possibilities. Under the null hypothesis of randomness, any rank arrangement from all possibilities should be equi-probable. Bartels Rank test is also suitable for small samples.

To get a feel of the test: consider two of the vignettes provided by the R package, `randtests`.

Example 5.1 in Gibbons and Chakraborti (2003), p.98.

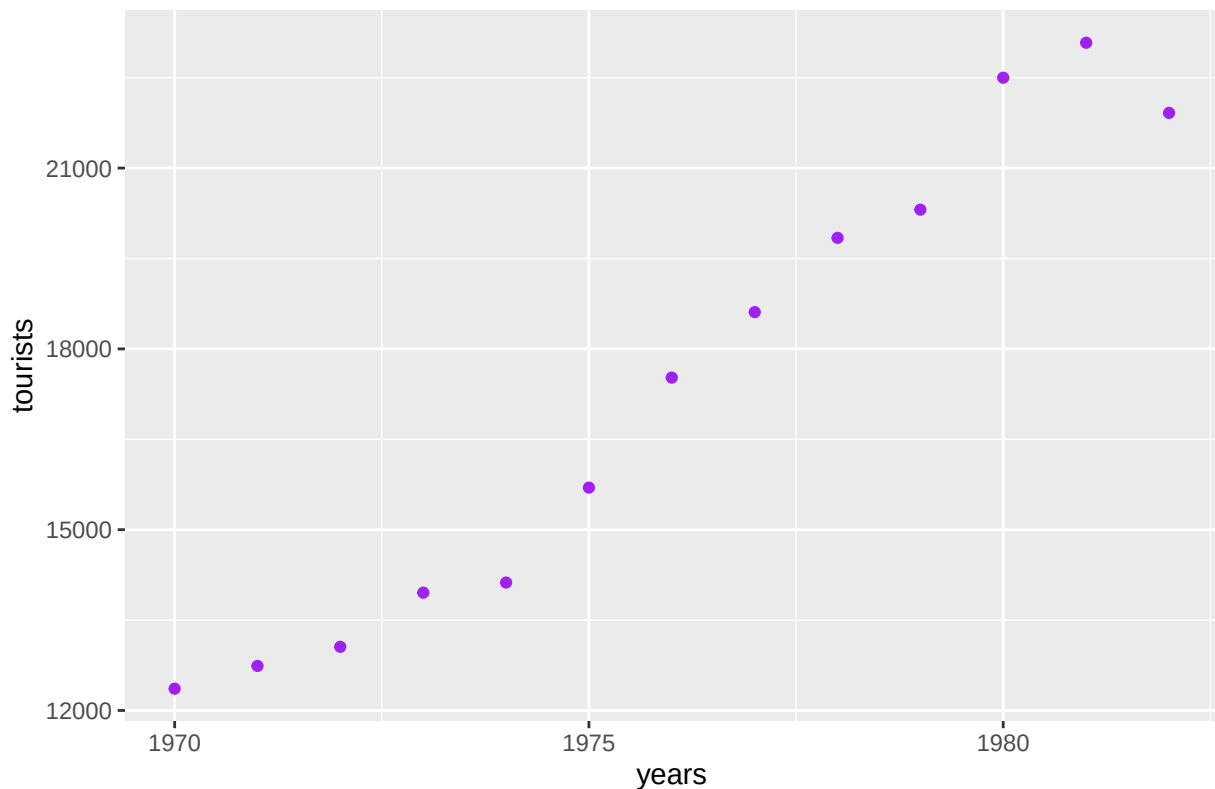
Annual data on total number of tourists to the United States for 1970-1982.

```
years <- seq(ymd("1970-01-01"), ymd("1982-01-01"), by = "years")
tourists <- c(
  12362, 12739, 13057, 13955, 14123, 15698, 17523, 18610, 19842,
  20310, 22500, 23080, 21916
)
ggplot(years, tourists, colour = I("purple")) +
  ggtitle("Tourists to United States (1970-1982)") +
```

```
theme(plot.title = element_text(hjust = 0.5)) +
scale_x_date()
```

```
## Warning: `qplot()` was deprecated in ggplot2 3.4.0.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was generated.
```

Tourists to United States (1970–1982)



```
bartels.rank.test(tourists, alternative = "left.sided", pvalue = "beta")
```

```
##
## Bartels Ratio Test
##
## data: tourists
## statistic = -3.6453, n = 13, p-value = 1.21e-08
## alternative hypothesis: trend
```

What this tells us about the sample data is there is strong evidence *against* the null hypothesis of randomness. Instead, it favours the alternative hypothesis, that of a trend.

Example in Bartels (1982)

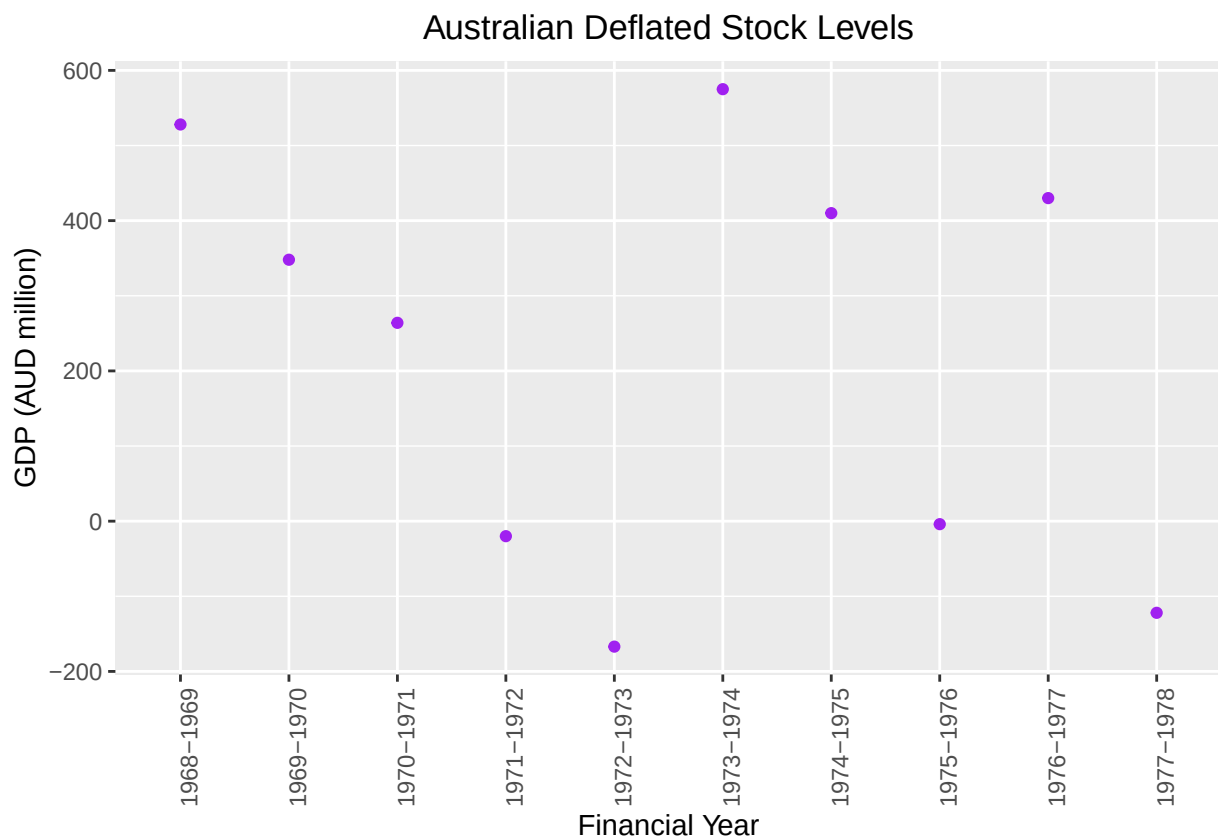
Changes in stock levels for 1968-1969 to 1977-1978 (in AUD million), deflated by the Australian gross domestic product (GDP) price index (base 1966-1967). Source

```
gdp <- c(528, 348, 264, -20, -167, 575, 410, -4, 430, -122)
df <- data.frame(
```

```

period = paste(sep = "-", 1968:1977, 1969:1978),
gdp = gdp,
stringsAsFactors = TRUE
)
ggplot(data = df, aes(period, gdp)) +
  geom_point(colour = I("purple")) +
  ggtitle("Australian Deflated Stock Levels") +
  theme(plot.title = element_text(hjust = 0.5)) +
  xlab("Financial Year") +
  theme(axis.text.x = element_text(angle = 90)) +
  ylab("GDP (AUD million)")

```



```
bartels.rank.test(gdp, pvalue = "beta")
```

```

##
##  Bartels Ratio Test
##
## data:  gdp
## statistic = 0.083357, n = 10, p-value = 0.9379
## alternative hypothesis: nonrandomness

```

Here, the sampled data provides weak evidence against the null hypothesis of randomness. Which does not support the alternative hypothesis of non-random data.

(For a simple guide on how to interpret the p-value, see [this](#))

Random Number Generators that can be used in Bash scripts

The following sections describe 3 RNG's. It includes a small description with typical use of the RNG. Then a list of 1000 values is produced and analysed using Bartels Rank test.

bash RANDOM variable

Bash, provides the shell variable \$RANDOM. It will generate a pseudorandom signed 16-bit integer between 0 and 32767.

RANDOM is easy to use in bash:

```
RANDOM=314
echo $RANDOM
```

```
## 1670
```

Here, we have seeded RANDOM with a value. Seeding a random variable will return the same sequence of numbers. This is required for results to be reproducible.

To generate a random integer between START and END use:

```
RANGE=$(( END - START + 1 ))
echo $(( (RANDOM % RANGE) + START ))
```

Where START < END are non-negative numbers.

For example, to simulate 10 rolls of a 6 sided dice:

```
START=1
END=6
RANGE=$(( END - START + 1 ))

RANDOM=314
for i in $(seq 10); do
    echo -n $(( (RANDOM % RANGE) + START )) " "
done
```

```
## 3 6 2 1 5 1 2 2 5 1
```

Checking Sequences from RANDOM The following code will generate a random sample using bash RANDOM. The sample will be scaled to 1. The generated sample will then be tested using Bartels Rank test, where the null hypothesis is that the sequence is random.

Prepare the test data:

```
RANDOM=314
# bc interferes with RANDOM
temprandom=$(mktemp temp.random.XXXXXXX)
for i in $(seq 1000); do
    echo "scale=2;$RANDOM/32768"
```

```
done > $temprandom
cat $temprandom | bc | awk '{printf "%0.2f\n", $0}' > bash.random
rm $temprandom
```

```
bash_text <- readLines(con <- file(paste0(getwd(), "/bash.random")))
close(con)
bash_random <- as.numeric(bash_text)
```

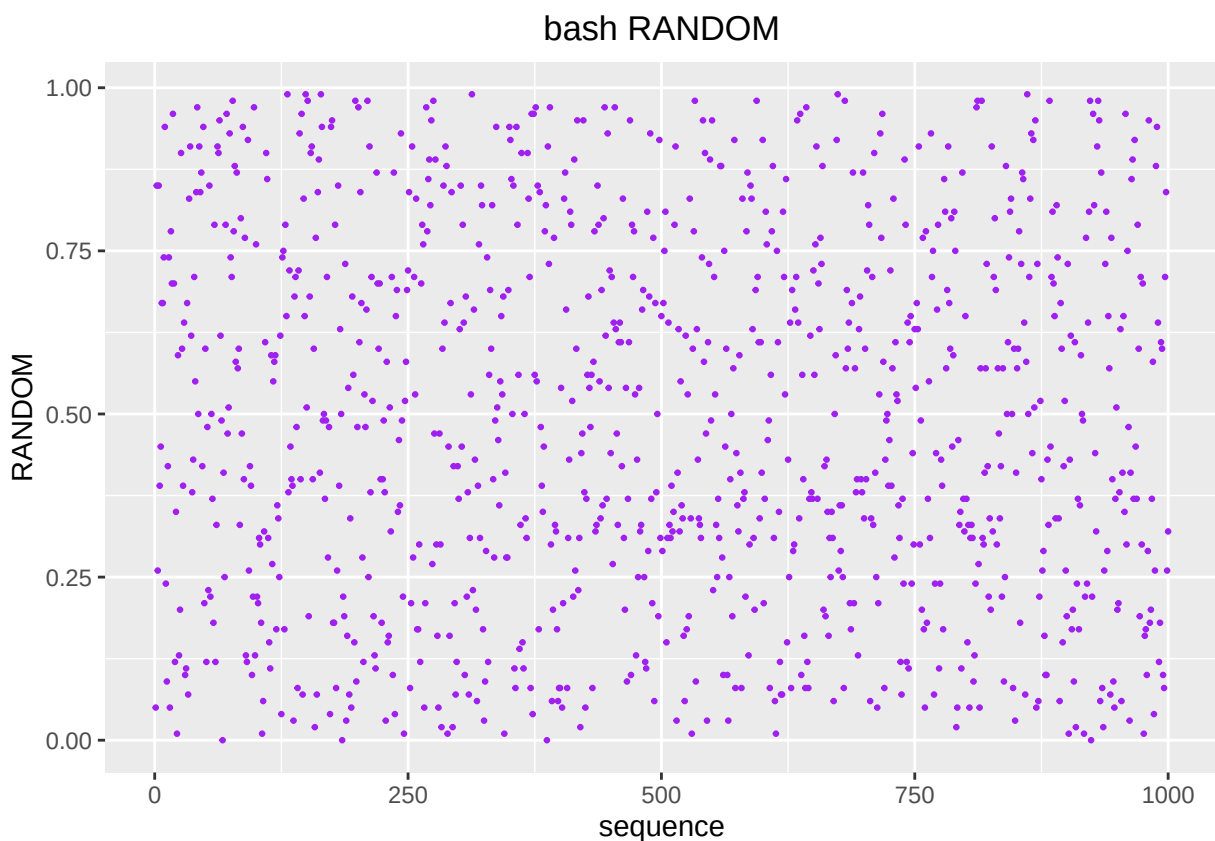
Show first 10 values:

```
head(bash_random, n = 10)
```

```
## [1] 0.05 0.85 0.26 0.85 0.39 0.45 0.67 0.67 0.74 0.94
```

Plot the sequence versus value from RNG:

```
bash_df <- data.frame(sequence = seq_along(bash_random), RANDOM = bash_random)
ggplot(bash_df, aes(x = sequence, y = RANDOM)) +
  geom_point(size = 0.5, colour = I("purple")) +
  ggtitle("bash RANDOM") +
  theme(plot.title = element_text(hjust = 0.5))
```



Run Bartels Rank test:

```
bartels.rank.test(bash_random, "two.sided", pvalue = "beta")
```

```
##
```

```
## Bartels Ratio Test
```

```
##
## data:  bash_random
## statistic = -1.6695, n = 1000, p-value = 0.09503
## alternative hypothesis: nonrandomness
```

Result With a p-value > 0.05 there is weak evidence against the null hypothesis of randomness.

awk rand()

The following code will generate a random sample using Here we are using `rand` function from GNU Awk. This function generates random numbers between 0 and 1. . The generated sample will then be tested using Bartels Rank test, where the null hypothesis is that the sequence is random.

Example where `rand()` seeded

```
echo | awk -e 'BEGIN {srand(314)} {print rand();}'
```

```
## 0.669965
```

If you don't specify a seed in `srand()` it will return the same results.

You can also generate random integers in a range.

For example, to simulate 10 rolls of a 6 sided dice:

```
echo | awk 'BEGIN {srand(314)} {for (i=0; i<10; i++) printf("%d ", int(rand() * 6 + 1));}
```

```
## 5 2 2 5 3 6 1 6 5 5
```

Checking Sequences from rand() Prepare the test data:

```
seq 1000 | awk -e 'BEGIN {srand(314)} {printf("%.2f\n",rand());}' > awk.random
```

```
awk_text <- readLines(con <- file(paste0(getwd(), "/awk.random"))
close(con)
awk_random <- as.numeric(awk_text)
```

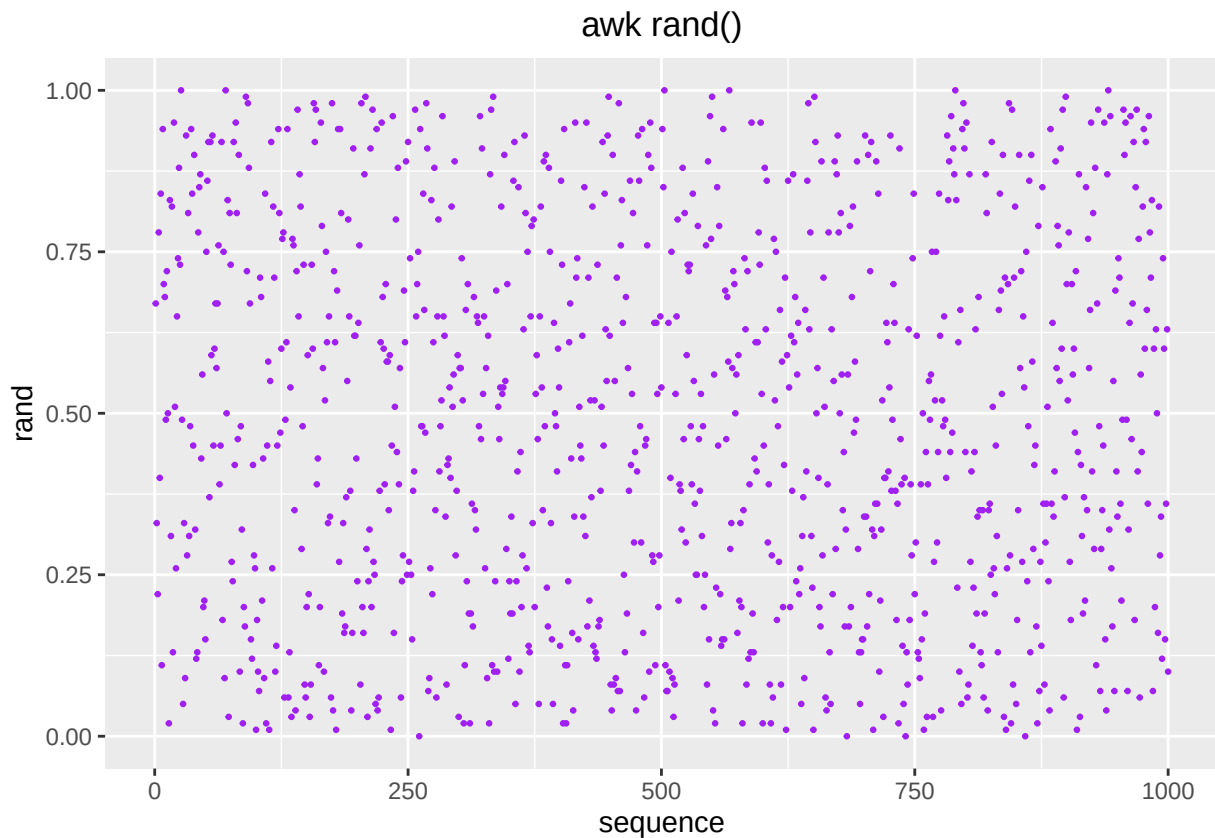
Show first 10 values:

```
head(awk_random, n = 10)
```

```
## [1] 0.67 0.33 0.22 0.78 0.40 0.84 0.11 0.94 0.70 0.68
```

Plot the sequence vs value from RNG:

```
awk_df <- data.frame(sequence = seq_along(awk_random), rand = awk_random)
ggplot(awk_df, aes(x = sequence, y = rand)) +
  geom_point(size = 0.5, colour = I("purple")) +
  ggtitle("awk rand()") +
  theme(plot.title = element_text(hjust = 0.5))
```



Run Bartels Rank test:

```
bartels.rank.test (awk_random, "two.sided", pvalue = "beta")
```

```
##
##  Bartels Ratio Test
##
## data:  awk_random
## statistic = 0.29451, n = 1000, p-value = 0.7685
## alternative hypothesis: nonrandomness
```

Result With a p-value > 0.05 there is weak evidence against the null hypothesis of randomness.

urandom device

The final tool we will look at is the `/dev/urandom` device. The device provides an interface to the kernels random number generator. This is a useful tool as it can generate a wide variety of data types.

For example print a list of unsigned decimal using `od(1)`:

```
seq 5 | xargs -I -- od -vAn -N4 -tu4 /dev/urandom
```

```
## 1808094704
## 123429207
## 2238733619
## 1678355517
```



```
## 2732492347
```

It can also be used to source random hexadecimal values:

```
seq 5 | xargs -I -- od -vAn -N4 -tx4 /dev/urandom
```

```
## cd06f3e4
## f535b3a1
## 9c176415
## ddbfca6d
## c89f71a8
```

Or it can generate a block of 64 random alphanumeric bytes using:

```
cat /dev/urandom | tr -dc 'a-zA-Z0-9' | fold -w 64 | head -n 5
```

```
## KwYr6Y94tRVWltExesKF663hum4KkZFald9RFLV9jDjuzkAxbMtrvMtQvSo6WNUC
## i4sLt7HJqYeRXTMiptLbcYkORoaiERvkf5yygVE4BX6aJWfmpwddX2TK0GyVXYEA
## m8Ufbp2vsMiTQ567kIdscxxyu81f7BITrY2UZAfttDkFQX04sA1o9Qf6RWCLVe7x
## ouMdOt5FSVdhvnaXXx871pUCM71IXUDqFZG2XQEiuundEXDl8IViIpvgrCvHmOiM
## nc9q5Ej9teCzi0zBQ6CIQ0i9tuDNvTC45M1lnxyGF8Z0u40soskafXnXwcmAB1sA
```

Checking Sequences from urandom To create a test sample that is scaled to 1, we will sample two digits from urandom and use these as decimal values. The generated sample will then be tested using Bartels Rank test, where the null hypothesis is that the sequence is random.

Prepare the test data:

```
cat /dev/urandom | tr -dc '0-9' | fold -w2 | \
  awk '{printf("0.%02d\n", $1)}' | head -1000 > urandom.random
```

```
ur_text <- readLines(con <- file(paste0(getwd(), "/urandom.random")))
close(con)
ur_random <- as.numeric(ur_text)
```

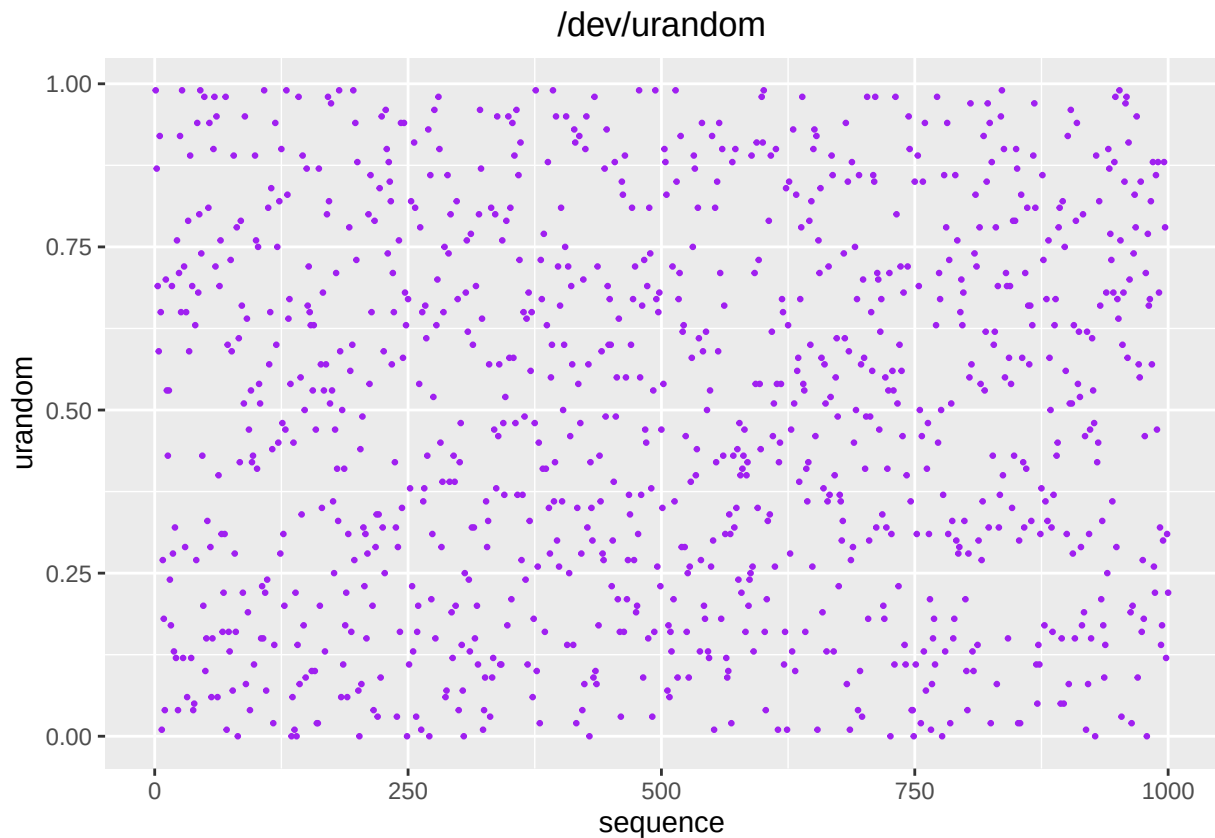
Show first 10 values:

```
head(ur_random, n = 10)
```

```
## [1] 0.99 0.87 0.69 0.59 0.92 0.65 0.01 0.27 0.18 0.04
```

Plot the sequence vs value from RNG:

```
urandom_df <- data.frame(sequence = seq_along(ur_random), urandom = ur_random)
ggplot(urandom_df, aes(x = sequence, y = urandom)) +
  geom_point(size = 0.5, colour = I("purple")) +
  ggtitle("/dev/urandom") +
  theme(plot.title = element_text(hjust = 0.5))
```



Run Bartels Rank test:

```
bartels.rank.test(ur_random, "two.sided", pvalue = "beta")
```

```
##
##  Bartels Ratio Test
##
## data:  ur_random
## statistic = -0.55031, n = 1000, p-value = 0.5824
## alternative hypothesis: nonrandomness
```

Result With a p-value > 0.05 there is weak evidence against the null hypothesis of randomness.

Some final thoughts

Of the tools explored, `urandom` is the most versatile. So it has broader application. The down side is, its results are not easily reproducible and issues have been identified by a study by Gutterman, Zvi; Pinkas, Benny; Reinman, Tzachy (2006-03-06) for the Linux kernel version 2.6.10.

Personally, this has been a useful learning exercise. For one it showed the limitations in generating and testing for (pseudo)random sequences. Indeed, Aaron Roth, has suggested:

As others have mentioned, a fixed sequence is a deterministic object, but you can still meaningfully talk about how “random” it is using Kolmogorov Complexity: (Kolmogorov complexity). Intuitively, a Kolmogorov random object is one that cannot be compressed. Sequences that are drawn from truly random sources are Kolmogorov random with extremely high probability.

Unfortunately, it is not possible to compute the Kolmogorov complexity of sequences in general (it is an undecidable property of strings). However, you can still estimate it simply by trying to compress the sequence. Run it through a Zip compression engine, or anything else. If the algorithm succeeds in achieving significant compression, then this certifies that the sequence is -not- Kolmogorov random, and hence very likely was not drawn from a random source. If the compression fails, of course, it doesn't prove that the sequence has high Kolmogorov complexity (since you are just using a heuristic algorithm, not the optimal (undecidable) compression). But at least you can certify the answer in one direction.

In light of this knowledge, let's run the compression tests for the sequences above:

```
ls -l *.random
```

```
## -rw-rw-r-- 1 frank frank 5000 2026-02-01 09:43 awk.random
## -rw-rw-r-- 1 frank frank 5000 2026-02-01 09:43 bash.random
## -rw-rw-r-- 1 frank frank 5000 2026-02-01 09:43 urandom.random
```

Compress using zip:

```
for z in *.random; do zip ${z%.random} $z; done
```

```
## updating: awk.random (deflated 71%)
## updating: bash.random (deflated 72%)
## updating: urandom.random (deflated 72%)
```

Compare this to non-random (trend) data:

```
for i in $(seq 1000); do printf "0.%02d\n" $(( i % 100 )) ; done > test.trend
zip trend test.trend
```

```
## updating: test.trend (deflated 96%)
```

Or just constant data:

```
for i in $(seq 1000); do echo 0.00 ; done > test.constant
zip constant test.constant
```

```
## updating: test.constant (deflated 99%)
```

So zipping is a good, if rough, proxy for a measure of randomness.

Stay tuned for part two to discover how random data can be used in testing code.